

PROJECT STEELBORE

CALIPER

Product Requirements Document

Version 2.0

High-Performance Raster-to-Vector Tracing Engine
Heterogeneous CPU + GPU + NPU Architecture

Document Date: 2026-04-30T00:00:00Z

Author: Mohamed Hammad

SPDX-License-Identifier: GPL-3.0-or-later
github.com/UnbreakableMJ/caliper

Forged in Steelbore

1. Executive Summary

Caliper is a high-performance, Rust-native raster-to-vector tracing engine governed by The Steelbore Standard v1.0 and the Steelbore Dual-Mode Self-Documenting CLI Framework (SFRS v1.0.0). It converts pixel-based raster images (PNG, JPEG, BMP, WebP, TIFF, AVIF, QOI, and more) into mathematically precise vector graphics (SVG, PDF, EPS, DXF) through a heterogeneous pipeline that exploits every available compute resource: multi-core CPUs with explicit SIMD vectorization (SSE2/SSE4.1/AVX2/AVX-512/NEON/SVE2), GPUs via portable compute shaders (wgpu), and NPUs via ONNX Runtime execution providers (CUDA, TensorRT, OpenVINO, DirectML, QNN, CoreML, Vitis AI).

The name Caliper reflects the tool's core philosophy: precision measurement and exacting tolerances applied to the inherently imprecise domain of pixel data. Caliper does not invent new tracing algorithms – it composes the best published ones (Suzuki-Abe contour following, O(n) spline fitting in the VTracer lineage, RANSAC primitive recognition, OKLab k-means quantization) under a novel adaptive meta-algorithm called the Assayer, which inspects each input and routes it to the optimal pipeline configuration.

Caliper ships as a single binary 'caliper' that is simultaneously a POSIX-compliant CLI, an interactive TUI (--format explore), an MCP server (caliper mcp), and an embeddable shared library (libcaliper). Every data-returning command supports --json with a stable schema. Every error carries a runnable hint. Every destructive operation supports --dry-run.

2. Project Identity

Attribute	Value
Project Name	Caliper
Domain	Raster-to-vector tracing / image vectorization
Naming Origin	Caliper – precision measuring instrument; metallurgical lineage
Binary Name	caliper
Crate Name	caliper-trace (root); workspace crates listed in §2.1
Library Name	libcaliper (C-ABI shared library via cdylib)
License	GPL-3.0-or-later (SPDX-compliant)
MSRV	Rust 1.82.0 (Edition 2024)
Tier-1 Targets	x86_64-unknown-linux-{gnu,musl}, aarch64-unknown-linux-gnu
Tier-2 Targets	x86_64-apple-darwin, aarch64-apple-darwin, x86_64-pc-windows-msvc
Tier-3 Targets	wasm32-unknown-unknown, riscv64gc-unknown-linux-gnu

2.1 Module Naming (Metallurgical Convention)

Crate	Role
caliper	Top-level binary (CLI + TUI dispatcher)
caliper-trace	Public library API surface (re-exports from internal crates)
caliper-crucible	Pipeline orchestrator – drives the seven-stage DAG
caliper-assayer	Adaptive pipeline router – analyzes input, selects algorithms
caliper-smelt	Color quantization (k-means in OKLab; median-cut; octree)
caliper-etch	Edge detection (Sobel/Scharr SIMD; HED/BDCN ML pathway)
caliper-burin	Contour tracing (Suzuki-Abe; connected-component labeling)
caliper-temper	Path fitting ($O(n)$ spline; corner detection; primitive RANSAC)
caliper-weld	Tile-boundary stitching and layer assembly
caliper-cast	Output encoding (SVG, PDF, EPS, DXF writers)
caliper-bellows	GPU compute backend (wgpu / WGSL compute shaders)
caliper-touchstone	NPU/ML backend (ort + ONNX Runtime execution providers)
caliper-anvil	SIMD kernels (std::simd portable + std::arch intrinsics)
caliper-tui	Ratatui interface for --format explore

The names are mnemonic: Crucible holds the molten work; the Assayer measures composition before refining; Smelt separates colors; Etch carves edges; Burin engraves contours; Temper hardens curves into final form; Weld joins boundaries; Cast pours the final mold; Bellows fans the GPU fire; Touchstone tests against learned patterns; Anvil shapes raw pixels.

3. Problem Statement

No existing FOSS raster-to-vector tool combines all of: native multi-color tracing, multi-tier hardware acceleration (CPU SIMD + GPU + NPU), comprehensive output format support, a scriptable POSIX-compliant CLI with structured JSON output, an MCP server surface, an interactive TUI, and a memory-safe, security-hardened implementation.

Tool	Strengths	Critical Deficiencies
Potrace	Reference quality on bi-level	Bi-level only; single-threaded; $O(n^2)$ fitting; no GPU/NPU; no SIMD; no JSON
AutoTrace	Color + centerline tracing	C codebase with memory safety issues; no SIMD; aging; brittle
VTracer	$O(n)$ fitting; native color; Rust	Limited concurrency; no GPU/NPU; no TUI; SVG-only; no agent CLI surface
Inkscape Trace	GUI integration	GUI-only; non-scriptable; multi-pass produces bloated output

ImageTracerJS	Browser-portable	Orders of magnitude slower than native; no concurrency model
Vectorizer.ai	AI shape fitting; symmetry	Closed-source SaaS; subscription-only; cloud-only; no offline mode
Vectorizer.io	Simple online tool	Closed-source SaaS; credit-based; no API for new users

Caliper closes this gap as the first FOSS tool to treat heterogeneous compute (CPU + GPU + NPU) as a first-class architectural concern, while maintaining strict POSIX compliance, memory safety, and full Steelbore Standard adherence.

4. Design Philosophy

Precision through measurement. Tracing quality is a measurable quantity, not a subjective judgment: sub-pixel path deviation, shape count, total path length, curve smoothness, and compression ratio are all instrumented and surfaced in real time.

Composition over invention. Caliper does not introduce a fundamentally new tracing algorithm. The published literature (Selinger's Potrace, the VTracer hierarchical clustering pipeline, Suzuki-Abe boundary following, RANSAC primitive fitting, OKLab perceptual quantization) is well-grounded and battle-tested. Caliper's contribution is an adaptive meta-algorithm – the Assayer (§7.2) – that dispatches to the optimal combination of these components for each specific input.

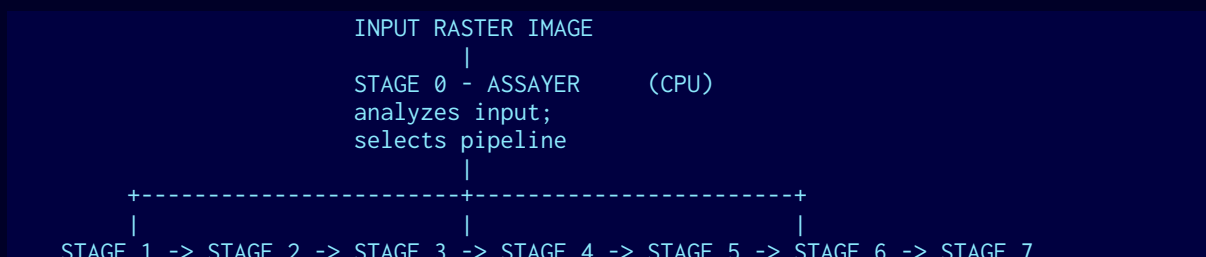
Hardware-conscious from the ground up. Three-tier compute (CPU + GPU + NPU) is not bolted on. The pipeline is designed as a series of stages with explicit declared compute affinities, and the runtime selects the best available backend per stage based on detected hardware capabilities and input characteristics.

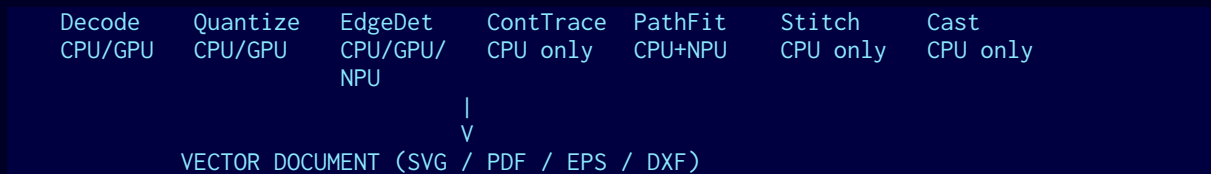
Two co-equal readers. Per the Steelbore Agentic CLI standard, every output is rendered twice: once for a human in a TTY (color, tables, TUI), once for an AI agent or pipe consumer (compact JSON, stable schema, runnable error hints). Neither rendering is a conversion of the other; both are first-class citizens.

Full Steelbore compliance. Every provision of the Steelbore Standard v1.0 applies: GPL-3.0-or-later, SPDX headers on every source file, ISO 8601 UTC timestamps, 24-hour time, metric-only units, WCAG 2.1 AA contrast, dual CUA+Vim keybindings, six-token color palette, Share Tech Mono / Inconsolata typography, no telemetry, no network access, local-storage default.

5. Architecture Overview

Caliper's pipeline is a directed acyclic graph (DAG) of seven stages, each with a declared compute affinity that determines which backends are eligible to execute it:





The Assayer (Stage 0) is a CPU-only routing stage that runs first on every invocation. Stages 1-3 are bandwidth-bound and embarrassingly parallel – these are eligible for GPU offload via Bellows. Stage 4 (contour tracing) is inherently sequential per contour but parallel across contours – CPU-only on Rayon. Stages 5-7 are CPU-only with optional NPU-assisted shape recognition in Stage 5 via Touchstone.

6. Performance Architecture

6.1 CPU Backend (Always Available)

6.1.1 Concurrency Model

Caliper's primary concurrency primitive is the Rayon work-stealing thread pool. The pipeline is expressed as parallel iterator chains that automatically distribute work across all available logical CPUs (`std::thread::available_parallelism()`). On NUMA architectures, Caliper respects CPU affinity masks to avoid cross-socket memory traffic.

Three-level parallelism is exploited within the CPU backend:

- ▶ **Layer-level** – after color quantization produces N color layers, each layer is traced concurrently ($\approx 16\times$ parallelism for typical 16-color images).
- ▶ **Tile-level** – each layer is spatially decomposed into 512×512 tiles (configurable via `--tile-size`). For a 4096×4096 image this yields 64 independent work units per layer.
- ▶ **Path-level** – within a single tile, disconnected contour regions are identified via union-find and fitted independently.

Theoretical parallelism for a 16-color, 4096×4096 image is $16 \times 64 \times K$ work units, where K is the average disconnected-contour count per tile. Rayon's work-stealing scheduler dynamically balances across all three levels, achieving near-linear scaling up to the physical core count.

Output assembly uses lock-free concurrent data structures (`crossbeam::SegQueue`); the final SVG/PDF assembly reads in deterministic z-order so output is reproducible regardless of thread scheduling.

6.1.2 SIMD Instruction Set Exploitation

Caliper uses Rust's `std::simd` (portable SIMD) for cross-architecture kernels and `std::arch` intrinsics for instruction-set-specific paths, with runtime dispatch selecting the optimal codepath:

Instruction Set	Architecture	Targeted Kernels
SSE2 / SSE4.1	x86_64 baseline	Color distance, edge detection, pixel blending

AVX2 + FMA3	x86_64 (Haswell+)	Color quantization, histogram, 8-wide pixel ops
AVX-512	x86_64 (Skylake-X+)	16-wide color clustering, masked boundary handling
NEON	aarch64	Color distance, edge kernels, quantization
SVE / SVE2	aarch64 (ARMv9+)	Scalable-width color processing
WASM SIMD128	wasm32	Browser/edge targets
RVV (RISC-V Vector)	riscv64gc (Tier 3)	Future – vector extension for V-spec hardware

Dispatch overhead is amortized to zero over the millions of pixel operations per tracing run. The build script (build.rs) emits cfg flags for compile-time gating, enabling LTO to eliminate dead codepaths on known targets. Build profiles release-portable (x86-64-v2), release-avx (x86-64-v3), and release-avx512 (x86-64-v4) target Microsoft's x86_64 microarchitecture levels.

6.1.3 Memory Architecture

- ▶ **Arena allocation** (bumpalo) for per-tile state – eliminates per-object allocation overhead and ensures cache-friendly spatial locality.
- ▶ **Structure-of-arrays (SoA) layout** – pixel data stored as separate R/G/B/A planes rather than interleaved RGBA. Maximizes SIMD register utilization. AoS→SoA transpose performed once at decode using SIMD shuffles.
- ▶ **Cache-resident tiling** – default 512×512 tile size (1 MB RGBA8) chosen to fit within L2 cache of modern x86_64 CPUs (1–2 MB per core). Configurable via --tile-size for systems with smaller or larger caches.

6.2 GPU Backend – Bellows (Optional)

The GPU backend is built on wgpu, providing portable compute shaders across Vulkan, Metal, DX12, OpenGL, and WebGPU from a single WGSL codebase. It is feature-gated behind --features gpu and enabled at runtime via --accelerator gpu or auto-selected by the Assayer when a capable GPU is detected.

Stages eligible for GPU offload:

Stage	Why GPU is a Win
AoS→SoA transpose	Pure parallel memory rearrangement; bandwidth-bound; perfect GPU workload
Denoising (NLM, bilateral)	Per-pixel window operations; embarrassingly parallel
OKLab color-space conversion	Per-pixel transform; SIMD-friendly on GPU
Color quantization (k-means)	Distance-to-centroid for millions of pixels per iteration; classic GPGPU workload
Histogram for palette autodetect	Atomic-add reduction; well-supported on GPU
Sobel/Scharr edge detection	3×3 separable convolution; 32-byte-wide SIMD on GPU

Morphological ops	Per-pixel local maximum/minimum; straightforward compute shader
Connected-component labelling	Label-equivalence parallel algorithm; GPU only for very large images

Stages NOT eligible for GPU:

Contour tracing (sequential per contour), path fitting (variable-length contour input with branchy control flow), tile stitching, and SVG assembly remain on CPU. After GPU preprocessing completes, only the per-layer binary masks transfer back to CPU – typically 30-60× smaller than the input image.

Critical design constraint: minimize PCIe traffic. Pixel data uploads to GPU once at decode time; all GPU stages (1, 2, 3) operate on resident textures; only quantized layer masks return to host memory. Use wgpu storage textures and bind groups to keep intermediate buffers GPU-resident.

6.3 NPU Backend – Touchstone (Optional)

The NPU backend uses the `ort Rust crate` (safe bindings to ONNX Runtime) with execution provider auto-selection. ONNX Runtime abstracts hardware details; Touchstone advertises preferred providers in priority order and falls back gracefully:

Hardware	Execution Provider	Operating System(s)
NVIDIA GPU (Tensor Cores)	TensorRTEExecutionProvider	Linux, Windows
NVIDIA GPU (CUDA)	CUDAExecutionProvider	Linux, Windows
Intel CPU/GPU/NPU	OpenVINOExecutionProvider	Linux, Windows
AMD Ryzen AI NPU	VitisAIExecutionProvider	Windows
Qualcomm Hexagon NPU	QNNExecutionProvider	Win on ARM, Android, Linux ARM
Apple Neural Engine	CoreMLExecutionProvider	macOS, iOS
Microsoft DirectML	DmlExecutionProvider	Windows (cross-vendor GPU)
Generic CPU fallback	CPUExecutionProvider	All platforms

ML-augmented stages:

- ▶ **ML edge detection (Stage 3 alternative).** Lightweight quantized-INT8 models – HED, BDCN, or compact distillations of EDTER – produce dramatically cleaner edges than Sobel on photographic or noisy input. Models are 2-8 MB quantized; inference is sub-100 ms on a 40+ TOPS NPU. Selected by the Assayer when input is classified as photographic or noise-degraded.
- ▶ **Semantic segmentation (v1.2.0+).** A small segmentation model identifies regions as text / line-art / fill / photographic / background, allowing per-region routing to the optimal tracing strategy. Replaces global pipeline configuration with region-aware one.
- ▶ **Super-resolution preprocessing (v1.2.0+).** For low-resolution inputs (<512px short edge), an INT8-quantized Real-ESRGAN or compact SwinIR model upscales 2-4× before tracing. Optional, off by default, opt-in via `--upscale`.

- **Shape classification (v1.2.0+).** After contour extraction, a tiny classifier (<1 MB) running on the NPU identifies geometric primitives more robustly than parametric RANSAC alone, especially on noisy or partial contours.

ML models are distributed as optional assets via 'caliper models pull', validated by BLAKE3 hash, and never auto-downloaded. PFA compliance is preserved: no automatic network access.

6.4 Hardware Capability Detection

At startup, Caliper enumerates available compute resources. Run:

```
caliper inspect --hardware --json
```

Emits a structured report of detected CPU SIMD levels, available GPU adapters (via `wgpu::Instance::enumerate_adapters`), and ONNX Runtime execution providers. The Assayer consumes this report to inform routing decisions. Override via `--accelerator <auto|cpu|gpu|npu|cpu+gpu|cpu+npu|all>`.

6.5 Benchmark Targets

Targets measured on Intel Core i7-14700K (8P+12E cores, 5.6 GHz boost), DDR5-5600, NVIDIA RTX 4070 (when GPU enabled), Intel NPU 4 (when NPU enabled):

Workload	CPU only	CPU + GPU	CPU + GPU + NPU
1080p PNG → SVG (16 colors)	< 200 ms	< 80 ms	< 60 ms
4K PNG → SVG (16 colors)	< 600 ms	< 220 ms	< 180 ms
8K PNG → SVG (16 colors)	< 2.0 s	< 700 ms	< 600 ms
Gigapixel BW scan → SVG	< 30 s	< 10 s	< 8 s
Photographic 4K (ML edges)	N/A	N/A	< 1.2 s
Batch 1000× 1080p PNGs	< 60 s	< 25 s	< 22 s
Scaling efficiency (20 cores)	≥ 85%	≥ 85%	≥ 85%
Peak memory (4K, 16 colors)	< 512 MB	< 768 MB	< 1024 MB

VTracer reference: ≈3.2 s for 4K. Potrace reference: ≈1.2 s for 1080p BW (single-threaded).

7. Algorithm Selection

7.1 Component Algorithms (No Reinvention)

Caliper composes published, well-grounded algorithms. None of them require reinvention.

Stage	Primary Algorithm	Rationale
Color quantization	k-means in OKLab + k-means++ init	Perceptually uniform; cheap; deterministic
Alt quantizers	Median-cut, octree	Available via <code>--quantizer</code>

Edge detection (classical)	Sobel / Scharr (3×3 separable)	SIMD-friendly; baseline quality
Edge detection (ML)	HED / BDCN / EDTER-distilled	Cleaner edges on photos and noisy input
Morphological ops	Erosion / dilation / opening / closing	Standard structuring-element operations
Boundary following	Suzuki-Abe (Theo Pavlidis alt)	Reference algorithm; robust hole detection
Contour simplification	Visvalingam-Whyatt, Douglas-Peucker	VW preserves visual character better
Corner detection	Angular threshold + curvature analysis	Configurable via <code>--corner-threshold</code>
Path fitting	O(n) least-squares Bézier (VTracer-style)	Linear time; superior to O(n ²)
Curve optimization	Selinger's segment-merging (Potrace §3.6)	Reduces output size losslessly
Primitive fitting	RANSAC (circle, ellipse, rect, star)	Embarrassingly parallel
Symmetry detection	Fourier descriptors + moment invariants	Mirror/rotational symmetry enforcement
Layer assembly	Stacking (default), Cutout (hierarchical)	Stacking yields more compact output

7.2 The Assayer – Adaptive Pipeline Router (Novel Contribution)

The Assayer is the only component of Caliper that does not exist in prior art. It is a CPU-only routing stage that runs first on every invocation and produces a `PipelineConfig` consumed by the Crucible orchestrator.

The Assayer analyzes the input image across multiple feature axes:

Feature	Method	Influence
Effective color count	Histogram + silhouette score	Quantizer + palette size
Edge density	Gradient magnitude statistics	Edge detector (classical vs ML)
Noise level	Local variance + frequency-domain	Denoising strength + ML preprocessing
Image class	Heuristic + (optional) NPU classifier	Logo, line-art, technical, photo, pixel art
Resolution	Dimensions + DPI metadata	Tile size + super-resolution opt-in
Color space	sRGB / Display P3 / Adobe RGB	Quantization color space
Hardware capabilities	From <code>caliper inspect --hardware</code>	Backend per stage

The output `PipelineConfig` declares per-stage choices:

```
{
  "image_class": "logo",
  "stages": {
    "preprocess": { "backend": "gpu", "denoise": false },
    "quantize": { "backend": "gpu", "algorithm": "kmeans_oklab", "k": 16 },
    "edge_detect": { "backend": "cpu", "algorithm": "sobel" },
    "trace": { "backend": "cpu", "algorithm": "suzuki_abe" },
    "fit": { "backend": "cpu", "algorithm": "spline_on", "primitives": true },
    "stitch": { "backend": "cpu" },
    "cast": { "backend": "cpu", "format": "svg" }
  },
  "rationale": "Low color count + sharp edges + small file size",
  "rationale": " -> classical CPU pipeline with primitive RANSAC enabled."
}
```

The Assayer is overrideable at every level: `--config <file>` accepts a user-authored PipelineConfig; per-stage flags (`--quantizer`, `--edge-detector`, `--accelerator`) override individual decisions. The Assayer's recommendation is always emitted in the JSON envelope's `metadata.assayer` field for reproducibility.

The Assayer is the closest thing Caliper has to a novel algorithm: not a single tracing algorithm, but an intelligent meta-algorithm that dispatches to the optimal combination of well-understood components. This is genuinely new – no existing FOSS tracer performs adaptive routing.

7.3 Image-Class Routing Table

Image Class	Quantizer	Edge Detector	Path Mode	Primitives	Symmetry
Logo / Icon	k-means OKLab	Sobel	spline	Yes (RANSAC)	Yes
Line Art	Bi-level (Otsu)	(skip)	spline	No	No
Technical Drawing	Bi-level (Otsu)	(skip)	centerline	No	No
Pixel Art	Exact palette	(skip)	pixel	No	No
Illustration	k-means OKLab	Sobel	spline	Optional	Optional
Photographic	k-means high-k	ML (HED)	spline	No	No
Map / GIS	k-means OKLab	Sobel	spline	No	No

8. Pipeline Stages (Detailed)

8.1 Stage 0 – Assayer

CPU-only. Analyzes the input image (after fast metadata parsing and 1/8-resolution thumbnail computation) to produce the PipelineConfig. Runtime budget: $\leq 2\%$ of total tracing time.

8.2 Stage 1 – Decode and Preprocess

Decode via the image crate (PNG, JPEG, BMP, WebP, TIFF, AVIF, QOI, ICO, TGA, PNM, DDS, OpenEXR, farbfeld). Optional preprocessing: denoising (NLM or bilateral), contrast enhancement (CLAHE), alpha premultiplication, AoS→SoA transpose, tile decomposition. Eligible backends: CPU, GPU (Bellows).

8.3 Stage 2 – Color Quantization

k-means in OKLab with k-means++ init (default). Alternatives: median-cut, octree. Palette size auto-detected via silhouette analysis or set by --colors N. Special mode --bw bypasses to adaptive thresholding (Sauvola or Otsu). Eligible backends: CPU (SIMD), GPU (Bellows compute shader).

8.4 Stage 3 – Edge Detection

Classical: SIMD-vectorized Sobel/Scharr. Connected-component labeling via union-find for path-level parallelism. ML alternative: HED/BDCN/EDTER-distilled via Touchstone NPU when input is classified as photographic. Eligible backends: CPU (SIMD), GPU (Bellows), NPU (Touchstone).

8.5 Stage 4 – Contour Tracing

Suzuki-Abe boundary following with hole detection and parent-child contour hierarchy. Visvalingam-Whyatt simplification with --simplify-tolerance (default 1.0 px). Corner detection via angular threshold (--corner-threshold, default 60 degrees). CPU-only by nature – independent contours run in parallel via Rayon.

8.6 Stage 5 – Path Fitting

O(n) Bézier fitting with SIMD-accelerated least-squares. Primitive RANSAC for circles, ellipses, rectangles, rounded rectangles, stars (in parallel – each trial is independent). Symmetry detection and enforcement. Curve optimization (Selinger segment merging). Path modes: pixel (no fitting), polygon (linear), spline (full Bézier). Optional Touchstone NPU shape classification. Eligible backends: CPU + (optional) NPU.

8.7 Stage 6 – Stitching and Assembly

Boundary stitching merges paths crossing tile boundaries via overlap regions. Layer stacking in z-order (stacked default, cutout hierarchical). Path optimization removes redundant points and merges adjacent same-color paths. CPU-only.

8.8 Stage 7 – Cast (Output Encoding)

Streaming writers per format. Configurable decimal precision (--path-precision, default 3). SVG 1.1 with optional CSS classes. PDF 1.7 with native vector paths. EPS 3.0 with proper BoundingBox. DXF R14 with layer-per-color. CPU-only.

9. CLI Interface (SFRS Compliance)

Caliper's CLI follows the Steelbore Dual-Mode Self-Documenting CLI Framework (SFRS v1.0.0) without exception.

9.1 Noun-Verb Command Structure

Command	Description
caliper trace <input> -o <output>	Trace single image to vector output
caliper batch <glob> -o <dir>	Batch-trace with parallelism across files
caliper inspect <input>	Analyze image; report optimal pipeline config
caliper inspect --hardware	Report detected CPU/GPU/NPU capabilities
caliper preview <input>	Quick low-res trace with metrics to stdout
caliper benchmark <input>	Trace with detailed performance profiling
caliper palette <input>	Extract and display auto-detected color palette
caliper config get set describe	View/modify persistent config
caliper models list pull verify	Manage Touchstone ML model assets
caliper schema [<command>]	Emit JSON Schema (Draft 2020-12) for outputs
caliper describe	Emit tool manifest (capabilities, sub-commands, version)
caliper mcp	Run as MCP server over stdio / sse / streamable-http

Standard verbs from SFRS §2 Rule 7: list, get, create, update, delete, apply, sync, describe, schema, plus trace, batch, inspect, preview, benchmark, palette, pull, verify as domain-specific verbs.

9.2 Global Flags (SFRS §3 – Identical Across All Steelbore CLIs)

Flag	Default	Description
--json	–	Alias for --format json
--format <fmt>	auto	json, jsonl, yaml, csv, explore
--fields <f1,f2,...>	all	Restrict output to listed fields
--dry-run	false	Emit action plan as JSON; no side effects
--verbose / -v	–	Diagnostic output to stderr
--quiet / -q	–	Suppress non-error stderr
--no-color	false	Disable ANSI color (= --color=never)
--color <when>	auto	never / always / auto
--help / -h	–	Help text with ≥2 examples per sub-command
--version	–	Version + build info
--absolute-time	false	Disable relative-time rendering in human mode
--print0 / -0	false	NUL-delimited output for xargs -0

<code>--yes / --force</code>	<code>false</code>	Skip confirmation in non-TTY mode
------------------------------	--------------------	-----------------------------------

9.3 Caliper-Specific Flags

Flag	Default	Description
<code>--colors <N></code>	<code>auto</code>	Palette size (2-256, or 'auto')
<code>--quantizer <ALGO></code>	<code>kmeans</code>	kmeans, median-cut, octree
<code>--mode <MODE></code>	<code>spline</code>	pixel, polygon, spline, centerline
<code>--bw</code>	<code>false</code>	Force bi-level tracing
<code>--threads <N></code>	<code>auto</code>	Worker thread count
<code>--tile-size <N></code>	<code>512</code>	Tile edge length in pixels
<code>--corner-threshold <DEG></code>	<code>60</code>	Minimum angle for corner detection
<code>--simplify-tolerance <PX></code>	<code>1.0</code>	Path simplification tolerance
<code>--path-precision <N></code>	<code>3</code>	Decimal places in SVG path data
<code>--filter-speckle <N></code>	<code>4</code>	Discard contours smaller than N pixels
<code>--hierarchical <MODE></code>	<code>stacked</code>	stacked or cutout
<code>--output-format <FMT></code>	<code>svg</code>	svg, pdf, eps, dxf
<code>--accelerator <BACKEND></code>	<code>auto</code>	auto, cpu, gpu, npu, cpu+gpu, cpu+npu, all
<code>--simd <LEVEL></code>	<code>auto</code>	auto, sse2, avx2, avx512, neon, sve
<code>--edge-detector <ALGO></code>	<code>auto</code>	auto, sobel, scharr, hed, bdcn
<code>--config <PATH></code>	<code>-</code>	Load PipelineConfig from file
<code>--upscale <N></code>	<code>1</code>	Pre-trace upscale factor via NPU (1, 2, 4)
<code>--primitives <BOOL></code>	<code>auto</code>	Enable/disable RANSAC primitive fitting
<code>--symmetry <BOOL></code>	<code>auto</code>	Enable/disable symmetry detection

9.4 Output Mode Detection Cascade (SFRS §5)

First match wins:

1. Explicit `--format <fmt>` or `--json ->` that mode unconditionally.
2. `AI_AGENT=1`, `AGENT=1`, or `CI=true ->` JSON, no color, no TUI.
3. `isatty(stdout) true ->` human mode with color.
4. `stdout piped/redirected ->` JSON mode.
5. Fallback `->` human mode.

`--format explore` requires a TTY; if `AI_AGENT` is set, fall back to JSON and warn on `stderr` – never trap an agent in a TUI loop.

9.5 JSON Output Envelope (SFRS §6)

Every `--json` response is a single valid JSON document:

```
{
```

```

"metadata": {
  "tool": "caliper",
  "version": "1.0.0",
  "command": "caliper trace logo.png -o logo.svg",
  "timestamp": "2026-04-30T14:30:00Z",
  "assayer": { "image_class": "logo", "stages": { ... } },
  "hardware": {
    "cpu": { "simd": "avx2" },
    "gpu": { "adapter": "NVIDIA RTX 4070", "backend": "vulkan" },
    "npv": { "provider": "TensorRTExecutionProvider" }
  },
  "pagination": null
},
"data": {
  "input": { "path": "logo.png", "width": 1024, "height": 1024 },
  "output": { "path": "logo.svg", "format": "svg", "bytes": 18432 },
  "metrics": {
    "shapes": 47, "total_path_length": 12483.2,
    "max_deviation_px": 0.42, "elapsed_ms": 187
  }
}
}

```

snake_case property names. No trailing commas. No comments. No BOM. No ANSI escapes. Compact (not pretty-printed) when stdout is non-TTY. Null fields omitted via `serde(skip_serializing_if = "Option::is_none")`.

9.6 Exit Codes (SFRS §4 Canonical Map)

Code	Meaning	Caliper-specific causes
0	Success	Trace completed; output written
1	General failure	Tracing pipeline error
2	Usage error	Bad arguments, malformed flags
3	Resource not found	Input file missing; ML model not present
4	Permission denied	Output path not writable
5	Conflict	Output exists without <code>--force</code>
10	Unsupported input format	File format not recognized or unsupported
11	Hardware backend unavailable	<code>--accelerator gpu</code> requested but no GPU detected
12	ML model error	Model load failed, hash mismatch, inference error
13	Configuration error	<code>--config <file></code> invalid

9.7 Structured Error with Tips-Thinking Hints

```

{
  "error": {
    "code": "HARDWARE_UNAVAILABLE",
    "exit_code": 11,
    "message": "GPU backend requested but no compatible adapter detected.",
    "hint": "Run 'caliper inspect --hardware --json' to see available",
      backends, or use '--accelerator cpu' to fall back.",
    "timestamp": "2026-04-30T14:30:00Z",
    "command": "caliper trace photo.jpg --accelerator gpu",
    "docs_url": "https://github.com/UnbreakableMJ/caliper/docs/hardware.md"
  }
}

```

```
}  
}
```

Per Steelbore Agentic CLI §3, hints are runnable commands, not prose. Canonical hint catalog lives at `assets/error-hint-catalog.json`.

9.8 Agent Environment Detection

Variable	Effect
AI_AGENT=1	JSON, no color, no TUI, --yes implicit, minimal stderr
AGENT=1	Same as AI_AGENT
CI=true	JSON, no color, no TUI, normal verbosity
CLAUDECODE=1	Informational only – recorded in <code>metadata.invoking_agent</code>
CURSOR_AGENT=1	Informational only
GEMINI_CLI=1	Informational only
TERM=dumb	No color, no TUI; do not infer agent intent
NO_COLOR	No color (per <code>no-color.org</code> standard)
FORCE_COLOR	Force color even when not detected

9.9 MCP Server Surface (caliper mcp)

Per SFRS §2 Rule 8 and Agentic §6, Caliper exposes an MCP server because its sub-command count exceeds 10. The server uses lazy schema loading: `tools/list` returns only tool names, one-line descriptions, and capability tags (`read`, `write`, `destructive`, `idempotent`). Full schemas load only on `tools/get`.

Transports: `stdio` (default), `sse`, `streamable-http`.

```
caliper mcp --transport stdio  
caliper mcp --transport sse --port 7421
```

9.10 Pipeline Composition

```
# Trace and pipe to SVG0 for further optimization  
caliper trace input.png --format svg | svg0 --input - --output optimized.svg
```

```
# Discover available files and batch-trace (Nushell-friendly)  
fd -e png . | caliper batch --stdin -o vectors/
```

```
# Capture metrics separately from output  
caliper trace photo.jpg --json 2>/dev/null > metrics.json  
caliper trace photo.jpg --output-format svg -o photo.svg
```

```
# Streaming JSONL for very large batches  
caliper batch '**/*.png' -o vectors/ --format jsonl |  
  jq -c 'select(.data.metrics.elapsed_ms > 1000)'
```

10. TUI Interface (--format explore)

Built on `ratatui` + `crossterm`. Activates in TTY mode; suppressed under `AI_AGENT` / `AGENT` / `CI` / `TERM=dumb`.

10.1 Layout

- ▶ Left pane: source raster (terminal half-block characters; Sixel/Kitty graphics protocol where supported).
- ▶ Right pane: live vector preview, rerasterized at terminal resolution as parameters change.
- ▶ Bottom pane: parameter sliders (color count, corner threshold, simplification tolerance, speckle filter, accelerator selection).
- ▶ Status bar: shape count, path length, max deviation, elapsed time, CPU/GPU/NPU utilization, active SIMD level, Assayer recommendation.

10.2 Dual CUA + Vim Keybindings (Standard §7)

Action	CUA	Vim
Navigate parameters	Tab / Shift+Tab	j / k
Adjust value	Arrow keys	h / l
Toggle preview	Ctrl+P	p
Export current result	Ctrl+S	:w
Cycle output format	Ctrl+F	f
Toggle metrics overlay	Ctrl+M	m
Zoom in / out	Ctrl+= / Ctrl+-	+ / -
Reset to defaults	Ctrl+R	R
Switch accelerator	Ctrl+A	a
Show Assayer report	Ctrl+I	g i
Quit	Ctrl+Q / Esc	:q

10.3 Steelbore Visual Theme (Standard §8)

Element	Color	Hex
Background	Void Navy	#000027
Primary text / values	Molten Amber	#D98E32
Section headers	Steel Blue	#4B7EB0
Active controls / success	Radium Green	#50FA7B
Warnings / errors	Red Oxide	#FF5C5C
Info / links / tips	Liquid Coolant	#8BE9FD
Borders / separators (dim)	Steel Blue	#4B7EB0

WCAG 2.1 AA contrast verified for every foreground/background pairing.

Reduced-motion preference honored. Alt-screen buffer used so terminal scrollbar is preserved on exit.

11. Library API

11.1 Rust API

```
use caliper_trace::{TracingConfig, trace, Backend, OutputFormat};
```

```
let config = TracingConfig::builder()
    .colors(16)
    .mode(PathMode::Spline)
    .accelerator(Backend::Auto)
    .simplify_tolerance(1.0)
    .build();

let img = image::open("logo.png")?;
let doc = trace(&img, &config)?;

doc.write_svg("logo.svg")?;
println!("{:?}", doc.metrics());
Public surface: caliper_trace::TracingConfig, caliper_trace::trace,
caliper_trace::VectorDocument with to_svg(), to_pdf(), to_eps(), to_dxf(),
paths(), palette(), metrics(). All errors return Result<T, CaliperError>.
```

11.2 C-ABI FFI

```
caliper_status_t caliper_trace_file(
    const char* input_path,
    const char* output_path,
    const caliper_config_t* config);

caliper_config_t* caliper_config_new(void);
void caliper_config_set_colors(caliper_config_t*, int);
void caliper_config_set_accelerator(caliper_config_t*, caliper_backend_t);
void caliper_config_free(caliper_config_t*);
All FFI functions are panic-safe (std::panic::catch_unwind at the boundary) and
return error codes rather than panicking across FFI.
```

12. Security Hardening (Standard §3.3)

- ▶ **Zero unsafe blocks in application code.** All unsafe usage confined to caliper-anvil (SIMD intrinsics) with exhaustive test coverage and Miri validation.
- ▶ Position-independent code (-Crelocation-model=pic) and LLVM CFI (-Zsanitizer=cfi) in release builds.
- ▶ Stack canaries (-Cstack-protector=strong).
- ▶ Hardened allocator: optional hardened_malloc or mimalloc with security features.
- ▶ Input validation (per Agentic §7 threat model): path canonicalization, allow-list validation, control-character rejection at parse time, bounds-checking against schema-declared min/max, no shell interpolation (argv arrays only).
- ▶ Image decoder fuzzing: continuous cargo-fuzz campaigns against all input decoders with maintained corpus.
- ▶ Maximum image dimensions enforced before allocation (default 32768×32768; --max-dimension to adjust).
- ▶ No network access (PFA §6): no telemetry, no update checks, no DNS resolution. ML models are downloaded only via explicit caliper models pull.
- ▶ Supply chain: cargo-vet + cargo-deny in CI. Minimal dependency tree. No proc-macro crates beyond clap and serde.

- ▶ PQC readiness: model signature verification uses BLAKE3 hashing; release artifacts signed with hybrid Ed25519 + ML-DSA-65.

13. Supported Formats

13.1 Input

PNG (8/16-bit, paletted, interlaced, APNG first frame), JPEG (with EXIF orientation, CMYK→sRGB), BMP (incl. OS/2 variants), WebP (lossy/lossless, animated first frame), TIFF (multi-page, 8/16/32-bit), AVIF (via dav1d or rav1d), QOI, TGA (incl. RLE), PNM (PBM/PGM/PPM), ICO, DDS (BCn compressed), OpenEXR (HDR tone-mapped), farbfeld.

13.2 Output

Format	Extension	Version	Notes
SVG 1.1	.svg	v1.0	Streaming XML writer; primary output
PDF 1.7	.pdf	v1.0	Native vector paths via pdf-writer
EPS 3.0	.eps	v1.1	Encapsulated PostScript with BoundingBox
DXF R14	.dxf	v1.1	AutoCAD-compatible; layer-per-color
AI v8	.ai	v1.2	Adobe Illustrator compatible
GeoJSON	.geojson	v1.2	GIS workflows
HPGL	.hpgl	v1.2	Pen plotters and CNC

14. Build and Distribution

14.1 Build Profiles

Profile	Optimizations	Use Case
dev	Debug assertions, no LTO	Development
release	opt-level=3, thin-LTO, strip	General distribution
release-native	release + target-cpu=native	Local builds
release-portable	release + target-cpu=x86-64-v2	Broad x86_64 (SSE4.2)
release-avx	release + target-cpu=x86-64-v3	AVX2 systems
release-avx512	release + target-cpu=x86-64-v4	AVX-512 systems

14.2 Distribution

- ▶ Static musl binaries (x86_64, aarch64 Linux)
- ▶ Nix flake + NixOS module (Lattice integration)

- ▶ Crates.io: caliper-trace
- ▶ AUR (Arch Linux)
- ▶ Flatpak, AppImage
- ▶ Homebrew (macOS)
- ▶ WASM build via wasm-bindgen + wasm-bindgen-rayon
- ▶ MSI installer (Windows)

14.3 CI/CD Matrix

Tier 1: x86_64-unknown-linux-{gnu,musl}, aarch64-unknown-linux-gnu. Tier 2: x86_64-apple-darwin, aarch64-apple-darwin, x86_64-pc-windows-msvc. Tier 3: wasm32-unknown-unknown, riscv64gc-unknown-linux-gnu.

Every CI run: cargo clippy --deny warnings, cargo test, cargo miri test (anvil), cargo fuzz (regression corpus), cargo deny check, cargo vet, criterion benchmarks with statistical regression alerts.

15. Testing Strategy

Category	Approach
Unit	Per-function coverage; SIMD vs scalar reference comparison
Integration	End-to-end trace with golden-file SVG comparison (normalized)
Fuzz	cargo-fuzz against decoders + tracing pipeline
Miri	All unsafe in caliper-anvil validated under Miri
Benchmark	Criterion micro + end-to-end at 1080p / 4K / 8K
Visual regression	insta snapshot testing for TUI
Cross-platform	CI matrix: Linux, macOS, Windows × x86_64, aarch64
JSON schema	Validate every command's output against caliper schema <command>
Exit codes	Test suite asserts canonical map adherence
Agent env detection	AI_AGENT, AGENT, CI, CLAUDECODE, CURSOR_AGENT, GEMINI_CLI
Idempotency	Same invocation twice -> same output (modulo timestamp)
Input validation	Path traversal, control chars, oversize images all rejected
Cross-shell roundtrip	POSIX sh, Bash 5+, Brush, Nushell 0.111+, PowerShell 7.6+, Ion
UTF-8	All output is UTF-8 without BOM; Windows console set to CP65001

16. Dependency Tree

Crate	Purpose	License
image	Raster image decoding (all formats)	MIT/Apache-2.0
rayon	Work-stealing parallelism	MIT/Apache-2.0
crossbeam	Lock-free data structures	MIT/Apache-2.0
bumpalo	Arena allocator for per-tile state	MIT/Apache-2.0
wgpu	GPU compute (Bellows backend)	MIT/Apache-2.0
bytemuck	Safe transmutation for GPU buffer uploads	MIT/Apache-2.0
ort	ONNX Runtime bindings (Touchstone backend)	MIT/Apache-2.0
ndarray	N-dimensional arrays for ML tensor I/O	MIT/Apache-2.0
clap	CLI argument parsing (derive)	MIT/Apache-2.0
ratatui	TUI framework	MIT
crossterm	Terminal backend for ratatui	MIT
pdf-writer	PDF output	MIT/Apache-2.0
quick-xml	Streaming XML writer for SVG	MIT
serde + serde_json	JSON output	MIT/Apache-2.0
jiff	ISO 8601 / UTC timestamps	MIT/Apache-2.0
tracing	Structured logging and instrumentation	MIT
rmcp	MCP server (Rust Model Context Protocol)	MIT
blake3	Model asset hashing	CC0 / Apache-2.0
criterion	Benchmark framework (dev)	MIT/Apache-2.0
cargo-fuzz	Fuzz testing (dev)	MIT/Apache-2.0

All dependencies are GPL-3.0-or-later compatible. Compatibility enforced by cargo-deny in CI.

17. Roadmap

17.1 v0.1.0 – Foundation (CPU-only)

- ▶ Caliper crate workspace with all 14 module crates scaffolded
- ▶ caliper-anvil SIMD kernels for SSE2, AVX2, NEON
- ▶ caliper-crucible orchestrator with seven-stage DAG
- ▶ caliper-assayer with rule-based image classification
- ▶ caliper-smelt k-means OKLab quantizer
- ▶ caliper-etch Sobel/Scharr edge detection
- ▶ caliper-burin Suzuki-Abe contour tracing
- ▶ caliper-temper O(n) spline fitting
- ▶ caliper-cast SVG output
- ▶ CLI: trace, inspect, preview, palette, schema, describe

- ▶ JSON envelope, structured errors, hint catalog
- ▶ AGENTS.md, CLAUDE.md, SKILL.md, CONTRIBUTING.md at repo root

17.2 v0.2.0 – Concurrency and TUI

- ▶ caliper-tui (--format explore) with dual CUA+Vim keybindings
- ▶ Three-level Rayon parallelism (layer × tile × path)
- ▶ caliper batch with cross-file parallelism
- ▶ AVX-512 and SVE2 SIMD kernels
- ▶ PDF output

17.3 v0.3.0 – GPU Acceleration (Bellows)

- ▶ caliper-bellows wgpu backend
- ▶ GPU-accelerated quantization, edge detection, morphology
- ▶ Hardware capability detection (caliper inspect --hardware)
- ▶ --accelerator flag with auto-selection

17.4 v0.4.0 – NPU Acceleration (Touchstone)

- ▶ caliper-touchstone ort/ONNX Runtime backend
- ▶ ML edge detection (HED/BDCN INT8-quantized)
- ▶ Execution provider auto-selection (CUDA, TensorRT, OpenVINO, QNN, CoreML, DirectML, Vitis AI)
- ▶ caliper models pull/list/verify commands
- ▶ Shape classification via NPU classifier

17.5 v0.5.0 – Format Expansion

- ▶ EPS and DXF output
- ▶ Geometric primitive RANSAC (circles, ellipses, rectangles, stars)
- ▶ Symmetry detection and enforcement
- ▶ Centerline tracing mode

17.6 v1.0.0 – General Availability

- ▶ All features above, hardened
- ▶ Full MCP server surface (caliper mcp)
- ▶ libcaliper C-ABI stabilization with cbindgen-generated headers
- ▶ WASM build via wasm-bindgen-rayon
- ▶ Complete cross-platform CI matrix
- ▶ Compliance audit complete (Standard \$13 + SFRS \$9)

17.7 v1.1.0 – Region-Aware Routing

- ▶ Touchstone semantic segmentation (text / line-art / fill / photo / background)
- ▶ Per-region pipeline configuration
- ▶ Super-resolution preprocessing (--upscale)

17.8 v1.2.0 – Extended Outputs

- ▶ AI (Adobe Illustrator) format
- ▶ GeoJSON for GIS workflows
- ▶ HPGL for pen plotters and CNC machines
- ▶ Plugin system for custom preprocessing/post-processing stages

17.9 v2.0.0 – Intelligence

- ▶ Trained Assayer model (replaces rule-based classifier)
- ▶ Batch optimization with shared palette across multiple inputs
- ▶ Differentiable tracing (gradient-based parameter tuning)
- ▶ Diffusion-curve output (research preview)

18. Compliance Checklist

Per Steelbore Standard §13 audit gate, this PRD satisfies:

- ▶ §2 Metallurgical naming applied to all 14 module crates
- ▶ §3.1 Memory safety: Rust-first; unsafe confined to caliper-anvil
- ▶ §3.2 Concurrency designed-in (Rayon three-level); benchmarks declared
- ▶ §3.3 PQC readiness for model signing; cargo-audit; CFI; ASLR
- ▶ §4 GPL-3.0-or-later; SPDX headers on all source files
- ▶ §5.1 POSIX-compliant CLI; platform extensions feature-flagged
- ▶ §6 PFA: no telemetry, no auto-network, local storage default
- ▶ §7 CUA + Vim keybindings in TUI
- ▶ §8 Steelbore palette in TUI; Void Navy background mandatory
- ▶ §9 Share Tech Mono / Inconsolata fonts (in TUI Sixel rendering)
- ▶ §10 Material Design principles in TUI; WCAG 2.1 AA contrast
- ▶ §11 ISO 8601 dates, 24h time, UTC, metric units throughout

SFRS §9 + Agentic §9 (CLI-specific gates)

- ▶ **BLOCKER** – ISO 8601 + UTC timestamps everywhere; UTF-8 without BOM; POSIX-first default output; --json on every data-returning command; stdout = data only, stderr = everything else; structured errors on stderr in machine mode; path canonicalization + allow-list validation; non-TTY destructive ops require --yes / --force; <tool> schema emits JSON Schema Draft 2020-12.
- ▶ **CRITICAL** – NO_COLOR honored; --dry-run on every write command; AI_AGENT=1 triggers JSON + no-color + non-interactive; every error response has

non-empty runnable hint; --fields honored on every list/get command; MCP server uses lazy schema loading; control-character rejection on string arguments.

- ▶ **MAJOR** – AGENTS.md, CLAUDE.md, SKILL.md, CONTRIBUTING.md at repo root; --format jsonl for streaming-eligible commands; JSON output omits null fields; JSON output is compact (not pretty) when stdout is non-TTY.

19. References

- ▶ The Steelbore Standard v1.0 – github.com/UnbreakableMJ/steelbore-standard
- ▶ Steelbore SFRS v1.0.0 – Dual-Mode Self-Documenting CLI Framework
- ▶ Steelbore Rust Guidelines – github.com/UnbreakableMJ/rust-guidelines
- ▶ Selinger, P. – Potrace: a polygon-based tracing algorithm (2003)
- ▶ VTracer – github.com/visioncortex/vtracer
- ▶ Suzuki, S. & Abe, K. – Topological Structural Analysis of Digitized Binary Images by Border Following (1985)
- ▶ Visvalingam, M. & Whyatt, J. D. – Line generalisation by repeated elimination of points (1993)
- ▶ Björn Ottosson – OKLab color space – bottosson.github.io/posts/oklab
- ▶ Rayon – docs.rs/rayon
- ▶ Rust Portable SIMD – doc.rust-lang.org/std/simd
- ▶ wgpu – wgpu.rs
- ▶ ort (ONNX Runtime Rust bindings) – ort.pyke.io
- ▶ ONNX Runtime Execution Providers – onnxruntime.ai/docs/execution-providers
- ▶ x86_64 microarchitecture levels – en.wikipedia.org/wiki/X86-64#Microarchitecture_levels
- ▶ Microsoft Pragmatic Rust Guidelines – microsoft.github.io/rust-guidelines
- ▶ Model Context Protocol (MCP) – modelcontextprotocol.io

Forged in Steelbore

END OF DOCUMENT